K-VLSI

SV TESTBENCH ARCHITECTURE PROGRAMS

NAME: DHANAPAL K

KVLSI ID: KVLSI2501061

Under the guidance of :

Shashi Kant Sharma

INDEX

Sl No.	CONTENTS	Page No.
1.	4x1 MUX	01 – 05
2.	Priority Encoder	06 – 10
3.	2:4 Decoder	11 – 15
4.	Comparator	16 – 20
5.	Full Adder	21 – 25
6.	D Flip Flop	26 – 31

TESTBENCH ARCHITECTURE

1. 4x1 MUX

- **Design:**

```
module mux_4_1(i,s,y);
input [0:3]i;
input [1:0]s;
output reg y;
 always@(i,s)
   begin
    case(s)
      2'b00: y=i[0];
      2'b01: y=i[1];
      2'b10: y=i[2];
      2'b11: y=i[3];
      default: y=1'bx;
    endcase
   end
endmodule
```

- **Interface:**

```
interface mux_4_1_intf;
logic [0:3]i;
logic [1:0]s;
logic y;
endinterface
```

- **Transaction:**

```
class transaction;
  rand bit [0:3]i;
  rand bit [1:0]s;
bit y;
function void display(string name);
$display("[%s]: I=%b, S=%b, Y=%b", name, i, s, y);
endfunction
endclass
```

- **Generator:**

```
class generator;
```

```
  transaction tr;
mailbox mb1;
function new(mailbox mb1);
this.mb1=mb1;
endfunction
task main();
repeat(10)
begin
tr=new();
assert(tr.randomize());
mb1.put(tr);
tr.display("gen");
end
endtask
endclass
```

- **Driver:**

```
class driver;
transaction tr;
mailbox mb1;
virtual mux_4_1_intf vif;
  function new(mailbox mb1, virtual mux_4_1_intf vif);
this.mb1=mb1;
this.vif=vif;
endfunction
task main();
repeat(10)
begin
mb1.get(tr);
vif.i<=tr.i;
vif.s<=tr.s;
#2;
tr.display("drv");
end
endtask
endclass
```

- **Monitor:**

```
class monitor;
transaction tr;
mailbox mb2;
virtual mux_4_1_intf vif;
  function new(mailbox mb2, virtual mux_4_1_intf vif);
this.mb2=mb2;
this.vif=vif;
endfunction
```

```
  task main();
repeat(10)
  begin
    tr=new();
#2;

tr.i=vif.i;
tr.s=vif.s;
tr.y=vif.y;
mb2.put(tr);
tr.display("mon");
   end
endtask
endclass
```

- **Scoreboard:**

```
class scoreboard;
transaction tr;
mailbox mb2;
  function new(mailbox mb2);
this.mb2=mb2;
endfunction
  task main();
repeat(10)
begin
 bit expected_y;
mb2.get(tr);

case(tr.s)
2'b00: expected_y=tr.i[0];
2'b01: expected_y=tr.i[1];
2'b10: expected_y=tr.i[2];
2'b11: expected_y=tr.i[3];
default: expected_y=1'bx;
endcase
  tr.display("scb");
  if(expected_y==tr.y)
$display("Test Passed");
else
$error("Test Failed");

end
endtask
endclass
```

- **Environment:**

```
`include "transaction.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"

class environment;
generator gen;
driver drv;
monitor mon;
scoreboard scb;
mailbox mb1;
mailbox mb2;
virtual mux_4_1_intf vif;

  function new(virtual mux_4_1_intf vif);
this.vif=vif;
mb1=new();
mb2=new();
gen=new(mb1);
drv=new(mb1,vif);
mon=new(mb2,vif);
scb=new(mb2);
endfunction
task run();
fork
gen.main();
drv.main();
mon.main();
scb.main();
join_any
endtask
endclass
```

- **Test:**

```
`include "environment.sv"
module test(mux_4_1_intf inf);
environment env;
initial
begin
env=new(inf);
env.run();
end
endmodule
```

- **Testbench:**

```
`include "interface.sv"
`include "test.sv"
module top;
mux_4_1_intf inf();
  mux_4_1 dut(.i(inf.i), .s(inf.s), .y(inf.y));
  test tb(inf);
initial
begin
$dumpfile("dump.vcd");
$dumpvars;
end
endmodule
```

- **EDA Playground Link:**

  https://www.edaplayground.com/x/BUTt

2. Priority Encoder

- **Design:**

```
module priority_encoder(i,y);
  input [7:0]i;
  output reg [2:0]y;
  always@(i)
    begin
    casex(i)
      8'b00000001:y=3'b000;
      8'b0000001x:y=3'b001;
      8'b000001xx:y=3'b010;
      8'b00001xxx:y=3'b011;
      8'b0001xxxx:y=3'b100;
      8'b001xxxxx:y=3'b101;
      8'b01xxxxxx:y=3'b110;
      8'b1xxxxxxx:y=3'b111;
      default:y=3'bxxx;
    endcase
  end
  endmodule
```

- **Interface:**

```
interface prior_intf;
 logic [7:0]i;
 logic [2:0]y;
endinterface
```

- **Transaction:**

```
class transaction;
rand bit [7:0]i;
bit [2:0]y;
function void display(string name);
  $display("[%s]: I=%b,Y=%b",name,i,y);
endfunction
endclass
```

- **Generator:**

```
class generator;
```

```
rand transaction tr;
mailbox mb1;

function new(mailbox mb1);
  this.mb1=mb1;
endfunction

//write main task
 task main();
   repeat(20)
    begin
      tr=new();
      assert(tr.randomize());
      mb1.put(tr);
      tr.display("gen");

   end
 endtask

endclass
```

- **Driver:**

```
class driver;

transaction tr;
mailbox mb1;
virtual prior_intf vif;

  function new(mailbox mb1,virtual prior_intf vif);
   this.mb1=mb1;
   this.vif=vif;
endfunction

task main();
  repeat(20)
    begin

      mb1.get(tr);

      vif.i<=tr.i;
    #2;
      tr.display("drv");
   end
endtask

endclass
```

- **Monitor:**

```
class monitor;
  transaction tr;
  mailbox mb2;
  virtual prior_intf vif;
  function new(mailbox mb2, virtual prior_intf vif);
    this.mb2=mb2;
    this.vif=vif;
  endfunction
  task main();
    repeat(20)
      begin
      tr=new();
    #2
    tr.i=vif.i;
    tr.y=vif.y;
    mb2.put(tr);
    tr.display("mon");
    end
  endtask
endclass
```

- **Scoreboard:**

```
class scoreboard;
  transaction tr;
  mailbox mb2;
  function new(mailbox mb2);
    this.mb2=mb2;
  endfunction
  task main();
    repeat(20)
      begin
        bit [2:0]expected_y;
        mb2.get(tr);
        casex(tr.i)
          8'b00000001:expected_y=3'b000;
          8'b0000001x:expected_y=3'b001;
          8'b000001xx:expected_y=3'b010;
          8'b00001xxx:expected_y=3'b011;
          8'b0001xxxx:expected_y=3'b100;
          8'b001xxxxx:expected_y=3'b101;
          8'b01xxxxxx:expected_y=3'b110;
          8'b1xxxxxxx:expected_y=3'b111;
          default: expected_y=3'bxxx;
         endcase
        tr.display("scb");
```

```
        if(expected_y==tr.y)
          $display("Test Passed");
        else
          $error("Test Failed");
      end
    endtask
  endclass
```

- **Environment:**

```
  `include "transaction.sv"
  `include "generator.sv"
  `include "driver.sv"
  `include "monitor.sv"
  `include "scoreboard.sv"
  class environment;
    generator gen;
    driver drv;
    monitor mon;
    scoreboard scb;
    mailbox mb1;
    mailbox mb2;
    virtual prior_intf vif;
    function new(virtual prior_intf vif);
      this.vif=vif;
      mb1=new();
      mb2=new();
      gen=new(mb1);
      drv=new(mb1,vif);
      mon=new(mb2, vif);
      scb=new(mb2);
    endfunction
    task run();
      fork
        gen.main();
        drv.main();
        mon.main();
        scb.main();
      join_any
    endtask
  endclass
```

- **Test:**

```
  `include "environment.sv"
  module test(prior_intf inf);
    environment env;
```

```
  initial
    begin
      env=new(inf);
      env.run();
    end
endmodule
```

- **Testbench:**

```
`include "interface.sv"
`include "test.sv"
module top;

//instantiate  interface
prior_intf inf();

//instantiate dut
  priority_encoder dut(.i(inf.i),.y(inf.y));

//instantiate test
test tb(inf);

initial begin
  $dumpfile("dump.vcd");
  $dumpvars;
end

endmodule
```

- **EDA Playground Link:**

  https://www.edaplayground.com/x/gKyi

3. 2:4 Decoder

- **Design:**

```
module dec_2_4(a,y);
  input [1:0]a;
  output [3:0]y;
  assign y[0]=(~a[1])&(~a[0]);
  assign y[1]=(~a[1])&(a[0]);
  assign y[2]=(a[1])&(~a[0]);
  assign y[3]=(a[1])&(a[0]);
endmodule
```

- **Interface:**

```
interface dec_2_4_intf;
  logic [1:0]a;
  logic [3:0]y;
endinterface
```

- **Transaction:**

```
class transaction;
  rand bit [1:0]a;
  bit [3:0]y;
  function void display(string name);
    $display("[%s]: A=%b, Y=%b", name, a, y);
  endfunction
endclass
```

- **Generator:**

```
class generator;
 rand transaction tr;
  mailbox mb1;
  event done;
  int count;
  function new(mailbox mb1);
    this.mb1=mb1;
  endfunction
  task main();
    repeat(count)
      begin
        tr=new();
        assert(tr.randomize());
        mb1.put(tr);
```

```
        tr.display("gen");
        @(done);
      end
  endtask
endclass
```

- **Driver:**

```
class driver;
  transaction tr;
  mailbox mb1;
//  int count;
  virtual dec_2_4_intf vif;
  function new(mailbox mb1, virtual dec_2_4_intf vif);
    this.mb1=mb1;
    this.vif=vif;
  endfunction
  task main();
    forever
      begin
        mb1.get(tr);
        vif.a<=tr.a;
        //vif.y<=tr.y;
        #2;
        tr.display("drv");
      end
  endtask
endclass
```

- **Monitor:**

```
class monitor;
  transaction tr;
  mailbox mb2;
  virtual dec_2_4_intf vif;
  function new(mailbox mb2, virtual dec_2_4_intf vif);
    this.mb2=mb2;
    this.vif=vif;
  endfunction
  task main();
    forever
      begin
        tr=new();
        #2;
        tr.a=vif.a;
        tr.y=vif.y;
        mb2.put(tr);
```

```
        tr.display("mon");
      end
    endtask
  endclass
```

- **Scoreboard:**

```
  class scoreboard;
    transaction tr;
    mailbox mb2;
    event done;
    int count;
    function new(mailbox mb2);
      this.mb2=mb2;
    endfunction
    task main();
    forever
      begin
      bit [3:0] expected_y;
      mb2.get(tr);

      expected_y[0] = (~tr.a[1]) & (~tr.a[0]);
      expected_y[1] = (~tr.a[1]) & ( tr.a[0]);
      expected_y[2] = ( tr.a[1]) & (~tr.a[0]);
      expected_y[3] = ( tr.a[1]) & ( tr.a[0]);

      tr.display("scb");

      if (expected_y === tr.y)
        $display("TEST PASSED");
      else
        $display("TEST FAILED");
        ->done;
        count++;
        if(count==10)
          $finish;
    end
  endtask
  endclass
```

- **Environment:**

```
  `include "transaction.sv"
  `include "generator.sv"
  `include "driver.sv"
  `include "monitor.sv"
  `include "scoreboard.sv"
  class environment;
```

```
  generator gen;
  driver drv;
  monitor mon;
  scoreboard scb;
  event done;
  mailbox mb1;
  mailbox mb2;
  virtual dec_2_4_intf vif;
  function new(virtual dec_2_4_intf vif);
   this.vif=vif;
   mb1=new();
   mb2=new();
   gen=new(mb1);
   drv=new(mb1,vif);
   mon=new(mb2,vif);
   scb=new(mb2);
   scb.done=done;
   gen.done=done;
  endfunction
  task run();
   fork
    gen.main();
    drv.main();
    mon.main();
    scb.main();
   join_any
  endtask
 endclass
```

- **Test:**

```
`include "environment.sv"
module test(dec_2_4_intf inf);
 environment env;
 initial
  begin
   env=new(inf);
   env.gen.count=10;
   env.run();
  end
endmodule
```

- **Testbench:**

```
`include "interface.sv"
`include "test.sv"
module top;
 dec_2_4_intf inf();
```

```
    dec_2_4 dut(.a(inf.a), .y(inf.y));
    test tb(inf);
    initial
      begin
        $dumpfile("dump.vcd");
        $dumpvars;
      end
  endmodule
```

- **EDA Playground Link:**

  https://www.edaplayground.com/x/D8xr

4. Comparator

- **Design:**

```
module comparator(a,b,lt,gt,eq);
  input a,b;
  output lt,gt,eq;
  assign gt = (a > b);
  assign lt = (a < b);
  assign eq = (a == b);
endmodule
```

- **Interface:**

```
interface comparator_intf;
  logic a,b;
  logic lt, gt,eq;
endinterface
```

- **Transaction:**

```
class transaction;
  rand bit a,b;
  bit lt,eq,gt;
  function void display(string name);
    $display("[%s]: A=%b, B=%b, LT=%b, EQ=%b, GT=%b", name, a, b, lt, eq, gt);
  endfunction
endclass
```

- **Generator:**

```
class generator;
  transaction tr;
  mailbox mb1;
  function new(mailbox mb1);
    this.mb1=mb1;
  endfunction
  task main();
    repeat(10)
      begin
        tr=new();
        assert(tr.randomize());
        mb1.put(tr);
        tr.display("gen");
      end
  endtask
```

```
endclass
```

- **Driver:**

```
class driver;
 transaction tr;
 mailbox mb1;
 virtual comparator_intf vif;
 function new(mailbox mb1, virtual comparator_intf vif);
  this.mb1=mb1;
  this.vif=vif;
 endfunction
 task main();
  repeat(10)
   begin
    mb1.get(tr);
    vif.a<=tr.a;
    vif.b<=tr.b;
    #2;
    tr.display("drv");
   end
 endtask
endclass
```

- **Monitor:**

```
class monitor;
 transaction tr;
 mailbox mb2;
 virtual comparator_intf vif;
 function new(mailbox mb2, virtual comparator_intf vif);
  this.mb2=mb2;
  this.vif=vif;
 endfunction
 task main();
  repeat(10)
   begin
    tr=new();
    #2;
    tr.a=vif.a;
    tr.b=vif.b;
    tr.lt=vif.lt;
    tr.eq=vif.eq;
    tr.gt=vif.gt;
    mb2.put(tr);
    tr.display("mon");
   end
 endtask
```

```
endclass
```

- **Scoreboard:**

```
class scoreboard;
  transaction tr;
  mailbox mb2;
  function new(mailbox mb2);
    this.mb2=mb2;
  endfunction
  task main();
    repeat (10)
      begin
      bit expected_eq, expected_gt, expected_lt;
      mb2.get(tr);  // Get transaction from monitor

      // Checker logic
      expected_eq = (tr.a == tr.b);
      expected_gt = (tr.a >  tr.b);
      expected_lt = (tr.a <  tr.b);
        tr.display("scb");

      // Comparison and result
      if ((expected_eq === tr.eq) &&
          (expected_gt === tr.gt) &&
          (expected_lt === tr.lt))
        begin
          $display("Test PASSED");
      end
        else
          begin
            $error("Test FAILED");
      end
     end
  endtask
endclass
```

- **Environment:**

```
`include "transaction.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
class environment;
  generator gen;
  driver drv;
```

```
  monitor mon;
  scoreboard scb;
  mailbox mb1;
  mailbox mb2;
  virtual comparator_intf vif;
  function new(virtual comparator_intf vif);
   this.vif=vif;
   mb1=new();
   mb2=new();
   gen=new(mb1);
   drv=new(mb1,vif);
   mon=new(mb2,vif);
   scb=new(mb2);
  endfunction
  task run();
   fork
    gen.main();
    drv.main();
    mon.main();
    scb.main();
   join_any
  endtask
 endclass
```

- **Test:**

```
 `include "environment.sv"
 module test(comparator_intf inf);
  environment env;
  initial
   begin
    env=new(inf);
    env.run();
   end
 endmodule
```

- **Testbench:**

```
 `include "interface.sv"
 `include "test.sv"
 module top;
  comparator_intf inf();
  comparator dut(.a(inf.a), .b(inf.b), .lt(inf.lt), .eq(inf.eq), .gt(inf.gt));
  test tb(inf);
  initial
   begin
```

```
        $dumpfile("dump.vcd");
        $dumpvars;
      end
    endmodule
```

- **EDA Playground Link:**

  https://www.edaplayground.com/x/hh9s

5. Full Adder

- **Design:**

```
module FA(a,b,cin,s,cout);
input a,b,cin;
output s,cout;
assign {cout,s} = a+b+cin;
endmodule
```

- **Interface:**

```
interface fa_intf;
logic a,b,cin,s,cout;
endinterface
```

- **Transaction:**

```
class transaction;
rand bit a,b,cin;
bit s,cout;
function void display(string name);
  $display("[%s]: A=%b, B=%b, C=%b, Sum=%b, Carry=%b",name,  a,b,cin,s,cout);
endfunction
endclass
```

- **Generator:**

```
class generator;
rand transaction tr;
mailbox mb1;
  event done;
  int count;
function new(mailbox mb1);
this.mb1=mb1;
endfunction
task main();
  repeat(count)
begin
tr=new();
assert(tr.randomize());
mb1.put(tr);
tr.display("gen");
  @(done);
end
endtask
```

```
endclass
```

- **Driver:**

```
class driver;
transaction tr;
mailbox mb1;
virtual fa_intf vif;
function new(mailbox mb1, virtual fa_intf vif);
this.mb1=mb1;
this.vif=vif;
endfunction
task main();
forever
begin
mb1.get(tr);
vif.a<=tr.a;
vif.b<=tr.b;
vif.cin<=tr.cin;
#2;
tr.display("drv");
end
endtask
endclass
```

- **Monitor:**

```
class monitor;
transaction tr;
mailbox mb2;
virtual fa_intf vif;
function new(mailbox mb2, virtual fa_intf vif);
this.mb2=mb2;
  this.vif=vif;
endfunction
task main();
forever
begin
#2;
tr=new();
tr.a=vif.a;
tr.b=vif.b;
tr.cin=vif.cin;
tr.s=vif.s;
tr.cout=vif.cout;
mb2.put(tr);
tr.display("mon");
end
```

```
endtask
endclass
```

- **Scoreboard:**

```
class scoreboard;
  transaction tr;
mailbox mb2;
  event done;
  int count;
function new(mailbox mb2);
this.mb2=mb2;
endfunction
task main();
forever
  begin
mb2.get(tr);
    tr.display("scb");


if({tr.cout, tr.s}==tr.a + tr.b + tr.cin)
$display("Test passed");
else
$error("Test failed");
   // tr.display("scb");
    ->done;
    count++;
    if(count==10)
      $finish;

end
endtask
endclass
```

- **Environment:**

```
`include "transaction.sv"
`include "driver.sv"
`include "generator.sv"
`include "monitor.sv"
`include "scoreboard.sv"
class environment;
generator gen;
monitor mon;
driver drv;
scoreboard scb;
  event done;
```

```
mailbox mb1;
mailbox mb2;
virtual fa_intf vif;
function new(virtual fa_intf vif);
this.vif=vif;
mb1=new();
mb2=new();
gen=new(mb1);
drv=new(mb1, vif);
mon=new(mb2,vif);
scb=new(mb2);
  scb.done=done;
  gen.done=done;
endfunction
task run();
fork
gen.main();
drv.main();
mon.main();
scb.main();
join_any
endtask
endclass
```

- **Test:**

```
`include "environment.sv"
module test(fa_intf inf);
environment env;
initial
begin
env=new(inf);
  env.gen.count=10;
env.run();
end
endmodule
```

- **Testbench:**

```
`include "interface.sv"
`include "test.sv"
module top;
```

```
fa_intf inf();
  FA dut(.a(inf.a), .b(inf.b), .cin(inf.cin), .s(inf.s), .cout(inf.cout));
test tb(inf);
initial
begin
$dumpfile("dump.vcd");
$dumpvars;
end
endmodule
```

- **EDA Playground Link:**

  https://www.edaplayground.com/x/kECP

6. D Flip Flop

- **Design:**

```
module dff(dff_if vif);

  always@(posedge vif.clk)// use interface variables in design itself
    begin
      if(vif.rst==1'b1)
        vif.dout <= 1'b0;
      else
        vif.dout <= vif.din;
    end
endmodule
```

- **Interface:**

```
interface dff_if;
logic clk,rst;
logic din;
logic dout;
endinterface
```

- **Transaction:**

```
class transaction;

  // 2 state variable since din/dout will be either 1 or 0 not 4 state logic
  rand bit din;
  bit dout; // no randomization for output

  function void display(input string tag);
    $display("[%0s] : din : %0b : dout : %0b",tag,din,dout);
  endfunction

  function transaction copy();// deep copy for transaction
    copy = new();
    copy.din = this.din;
    copy.dout = this.dout;
  endfunction

endclass
```

- **Generator:**

```
  class generator;
```

```
transaction tr;
mailbox #(transaction) mbx;
mailbox #(transaction) mbxref;
event sconext;
event done;
int count;


function new( mailbox #(transaction) mbx,  mailbox #(transaction) mbxref);
 this.mbx = mbx;
 this.mbxref = mbxref;
  tr = new();
endfunction


task run();
  repeat(count) begin
    for(int i = 0; i<10; i++) begin
      assert(tr.randomize) else $error("[GEN] : RANDOMIZATION FAILED");
      mbx.put(tr.copy);
      mbxref.put(tr.copy);
      tr.display("GEN");
      @(sconext);
    end
  end
  ->done;
endtask
endclass
```

- **Driver:**

```
class driver;

  transaction tr;
  mailbox #(transaction) mbx;
  virtual dff_if vif;

  function new( mailbox #(transaction) mbx);
   this.mbx = mbx;
  endfunction

  task reset();
    vif.rst <= 1'b1;
    repeat(5) @(posedge vif.clk);
    vif.rst <= 1'b0;
    @(posedge vif.clk);
    $display("[DRV] : RESET DONE");
  endtask
```

```
  task run();
   forever begin
    mbx.get(tr);
    vif.din <= tr.din;
    tr.display("DRV");
    @(posedge vif.clk);
   end
   endtask
endclass
```

- **Monitor:**

```
class monitor;

 transaction tr;
 mailbox #(transaction) mbx;
 virtual dff_if vif;

 function new( mailbox #(transaction) mbx);
  this.mbx = mbx;
 endfunction

 task run();
   tr = new();
   forever begin
     @(posedge vif.clk);
     @(posedge vif.clk);
     tr.dout = vif.dout;
     mbx.put(tr);
    tr.display("MON");
   end
 endtask
endclass
```

- **Scoreboard:**

```
class scoreboard;

 transaction tr;
 transaction trref;
 mailbox #(transaction) mbx;
 mailbox #(transaction) mbxref;
 event sconext;


 function new( mailbox #(transaction) mbx,  mailbox #(transaction) mbxref);
  this.mbx = mbx;
```

```
 this.mbxref = mbxref;
endfunction

task run();
 forever begin
  mbx.get(tr);
  mbxref.get(trref);
  tr.display("SCO");
  trref.display("REF");
  if(tr.dout == trref.din)
   $display("[SCO] : DATA MATCHED");
  else
   $display("[SCO] : DATA MISMATCHED");
  ->sconext;
 end
endtask
endclass
```

- **Environment:**

```
class environment;

  generator gen;
  driver drv;
  monitor mon;
  scoreboard sco;

  event next;  /// gen -> sco

 mailbox #(transaction) gdmbx; ///gen - drv
 mailbox #(transaction) msmbx;  /// mon - sco
 mailbox #(transaction) mbxref; ///// gen -> sco

  virtual dff_if vif;

 function new(virtual dff_if vif);
  gdmbx = new();
  mbxref = new();

  gen = new(gdmbx, mbxref);
  drv = new(gdmbx);


  msmbx = new();
  mon = new(msmbx);
  sco = new(msmbx, mbxref);

  this.vif = vif;
  drv.vif = this.vif;
```

```
    mon.vif = this.vif;

    gen.sconext = next;
    sco.sconext = next;
  endfunction

  task pre_test();
    drv.reset();
  endtask

  task test();
  fork
    gen.run();
    drv.run();
    mon.run();
    sco.run();
  join_any
  endtask

  task post_test();
    wait(gen.done.triggered);
    $finish();
  endtask

  task run();
    pre_test();
    test();
    post_test();
  endtask
endclass
```

- **Testbench:**

```
`include "interface.sv"
`include "transaction.sv"
`include "generator.sv"
`include "monitor.sv"
`include "driver.sv"
`include "scoreboard.sv"
`include "environment.sv"

module tb;
  dff_if vif();
  dff dut(vif);
  initial begin
    vif.clk = 1'b0;
  end
```

```
  always #10 vif.clk <= ~vif.clk;

  environment env;

  initial begin
    env = new(vif);
    env.gen.count = 20;
    env.run();
  end

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
  end
endmodule
```

- **EDA Playground Link:**

  https://www.edaplayground.com/x/aFQL